



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

PROJECT REPORT

Submitted for

HACK-O-WEEK PROGRAM

“Browser History Management using Queue”

**A project report submitted in partial fulfillment of the requirements for the degree
of Bachelor of Technology in Computer Science and Engineering**

BACHELOR OF TECHNOLOGY COMPUTER SCIENCE AND ENGINEERING

Submitted By

NEHAL SAAD : 25070521071

ARNAV SAHU : 25070521072

AAREFA ADIL : 25070521073

UNDER THE GUIDANCE OF

PIYUSH CHAUHAN

Designation – Associate Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
AY 2025–26



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

CERTIFICATE

This is to certify that the Project work entitled "Browser History Management using Queue" is carried out by **Nehal Saad, Arnav Sahu and Aarefa Adil** in partial fulfillment for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**, Symbiosis International (Deemed University), Pune during the academic year 2025-2026.

Mentor name :PIYUSH CHAUHAN

Designation: Associate Professor

DECLARATION

I hereby declare that the project titled "Contact Book Search using Binary Search" submitted to Symbiosis Institute of Technology, Constituent of Symbiosis International (Deemed University) Pune for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a result of original research carried out by me. I understand that my report may be made electronically available to the public. It is further declared that the project report or any part thereof has not been previously submitted to any University or Institute for the award of any degree or diploma.

Name of Student with pin:

NEHAL SAAD : 25070521071
ARNAV SAHU : 25070521072
AAREFA ADIL : 25070521073

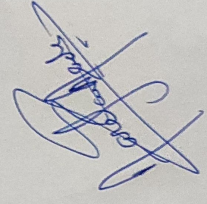
Degree: Bachelor of Technology in CSE

Department: Computer Science and Engineering

Title of the project: "Contact book search using binary search"

Nehal Saad

Sahu



Date: 11/04/26

ABSTRACT

This project presents a structured approach to browser history management using the Queue data structure. Traditional browser history systems often face inefficiencies in handling sequential page visits, especially when managing limited storage and ensuring smooth navigation. The primary objective of this project is to model browser history behavior using a First-In-First-Out (FIFO) mechanism, where the oldest visited pages are removed first as new pages are added.

In this implementation, each visited webpage is inserted into a queue, maintaining the order of access. When the predefined history limit is exceeded, the earliest entry is automatically deleted, ensuring optimized memory usage. This approach provides a simplified and predictable method for tracking user navigation patterns. Although real-world browsers typically use a combination of data structures, this project focuses on demonstrating the practical application of queues in managing linear browsing history.

The outcome of this system highlights efficient history tracking, reduced complexity, and improved performance in constrained environments. This project effectively illustrates how fundamental data structures like queues can be applied to solve real-world problems in browser functionality and data management.

Keywords:

- Browser History Management
- Queue Data Structure
- FIFO (First-In-First-Out)
- Data Structures
- Navigation System
- Web Browsing
- History Tracking
- Memory Optimization
- Enqueue Operation
- Dequeue Operation
- Algorithm Design
- Software Implementation
- User Navigation Behavior
- Linear Data Structure
- Performance Efficiency

TABLE OF CONTENTS

	Page
Certificate	2
Declaration	3
Abstract	4
Table of Contents	6
CHAPTER 1: BACKGROUND AND TECHNICAL OVERVIEW	7
CHAPTER 2: PROBLEM STATEMENT AND MOTIVATION	8
CHAPTER 3: NOVELTY AND INNOVATIVE CONTRIBUTIONS	10
CHAPTER 4: TECHNICAL ADVANTAGES AND PRACTICAL USEFULNESS	11
CHAPTER 5: DETAILED METHODOLOGY / SYSTEM ARCHITECTURE	12
CHAPTER 6: PRIOR ART AND RELATED WORK (Literature Survey)	15
CHAPTER 7: APPLICATIONS AND DEPLOYMENT AREAS	17
CHAPTER 8: CONCLUSION AND FUTURE SCOPE	19
REFERENCES (in IEEE format only)	21
APPENDICES	23

Chapter 1

Introduction

With the rapid growth of internet usage, web browsers have become essential tools for accessing and managing information. One of the fundamental features of any browser is the ability to maintain a history of visited web pages, allowing users to revisit previously accessed content. Efficient browser history management is crucial for ensuring smooth navigation, quick access, and optimal use of system resources.

Traditional browser systems use complex mechanisms to handle history navigation, including forward and backward traversal. However, in simplified models or constrained environments, managing this history efficiently becomes a challenge. This project focuses on implementing browser history management using a Queue data structure, which follows the First-In-First-Out (FIFO) principle. In this approach, the earliest visited webpage is removed first when the storage limit is exceeded, making it suitable for maintaining a fixed-size history.

The use of a queue provides a straightforward and efficient way to manage sequential browsing data, ensuring that memory usage remains controlled while preserving the order of page visits. This project demonstrates how fundamental data structures can be applied to solve practical problems in software systems, particularly in simulating browser behavior.

Chapter 2

PROBLEM STATEMENT AND MOTIVATION

Problem Statement

Efficient management of browser history is a critical aspect of web navigation systems.

As users continuously visit multiple web pages, maintaining an organized and memory-efficient record of these pages becomes challenging, especially in systems with limited storage capacity. Traditional methods of storing browser history may lead to excessive memory usage, redundancy, and reduced performance when handling large volumes of data.

The problem addressed in this project is to design and implement a simplified browser history management system that can store and update visited pages efficiently while maintaining a fixed size. The system must ensure that the oldest visited pages are automatically removed when new pages are added beyond the storage limit, thereby optimizing memory usage and maintaining the correct sequence of visits.

To solve this problem, an appropriate data structure is required that can handle sequential data insertion and deletion efficiently. This project utilizes the Queue data structure based on the First-In-First-Out (FIFO) principle to manage browser history in a structured and predictable manner, ensuring efficient performance and controlled memory utilization.

Motivation

The increasing reliance on web browsers for daily activities such as learning, communication, and information retrieval has made efficient browser history

management an important aspect of user experience. As users navigate

through numerous web pages, maintaining an organized and accessible record of visited sites becomes essential. However, handling large volumes of browsing data can lead to inefficiencies in storage and retrieval if not managed properly.

The motivation behind this project is to explore how fundamental data structures can be applied to solve real-world problems in a simple and efficient manner. By using the Queue data structure, this project aims to demonstrate a systematic approach to managing browser history based on the First-In-First-Out (FIFO) principle. This ensures that outdated or less relevant data is automatically removed, thereby optimizing memory usage.

Additionally, this project is motivated by the need to bridge theoretical concepts of data structures with practical implementation. It provides a clear understanding of how queues can be utilized in software systems, encouraging the development of efficient and scalable solutions for real-time applications.

Chapter 3

NOVELTY AND INNOVATIVE CONTRIBUTIONS

This project introduces a simplified yet effective approach to browser history management by applying the Queue data structure in a practical context. While modern browsers rely on complex hybrid models, this work focuses on a minimal and structured FIFO-based system, making the concept easy to understand, implement, and analyze.

The key novelty lies in modeling browser history using a fixed-size queue, where automatic removal of the oldest entries ensures efficient memory utilization without requiring manual intervention. This approach highlights how a basic linear data structure can be adapted to handle real-world problems in a resource-constrained environment.

Another innovative contribution of this project is its emphasis on educational clarity and system design simplicity. By isolating the history management mechanism and implementing it using queue operations such as enqueue and dequeue, the project provides a clear demonstration of data structure behavior in an applied scenario. This makes it particularly useful for academic purposes and for building foundational understanding.

Additionally, the project opens scope for further enhancements, such as integrating hybrid data structures (Queue + Stack) or extending the system to graphical user interfaces, thereby serving as a base model for more advanced browser simulation systems.

Chapter 4

TECHNICAL ADVANTAGES AND PRACTICAL USEFULNESS

Technical Advantages

- **Efficient**
Queue operations such as enqueue and dequeue operate in $O(1)$ time, ensuring fast insertion and deletion of browser history entries.
- **Optimized**
The fixed-size queue automatically removes the oldest entries, preventing memory overflow and ensuring controlled storage usage.
- **Simplicity**
The FIFO-based structure provides a clean and predictable mechanism for managing sequential data without complex logic.
- **Deterministic**
The system ensures consistent performance, as the oldest data is always removed first, avoiding ambiguity in data handling.
- **Scalability**
Suitable for lightweight applications where resources such as memory and processing power are limited.

Practical Usefulness

- **Browser History**
Can be used as a simplified model for storing and managing recently visited web pages.
- **Cache Management**
Applicable in scenarios where old data must be replaced with new data in a structured manner.
- **Embedded and Low-Resource**
Useful in devices with limited memory where efficient data handling is required.
- **Learning and Educational Tool**
Helps students understand real-world applications of queue data structures through practical implementation.
- **Foundation for Advanced Systems**
Acts as a base model for developing more complex navigation systems by integrating additional data structures like stacks.

CHAPTER 5

DETAILED METHODOLOGY / SYSTEM ARCHITECTURE

• 1. System Overview

The proposed system models browser history management using a **Queue data structure** based on the **FIFO (First-In-First-Out)** principle. The system stores visited web pages sequentially and maintains a fixed-size history by automatically removing the oldest entries when new pages are added beyond the defined limit.

- **2. System Components**
 - **User Interface (Input Layer):**
 - **User**
Accepts user actions such as visiting a new webpage.
 - **Processing Unit (Logic Layer):**
Implements queue operations (enqueue and dequeue) to manage history.
 - **Data Storage (Queue):**
Stores the sequence of visited web pages in order.
 - **Output Layer:**
Displays the current browser history to the user.

• 3. Working Methodology

The system operates in the following steps:

1. **Initialization:**
 - Define a queue with a fixed maximum size (history limit).
2. **Visiting a New Page:**
 - When a user visits a webpage, it is added to the queue using the enqueue operation.
3. **Handling Overflow:**
 - If the queue reaches its maximum capacity, the oldest webpage (front element) is removed using dequeue.
4. **Updating History:**

- The queue always reflects the most recent pages within the defined limit.

5. Displaying History:

- The system outputs the current contents of the queue in order of access.

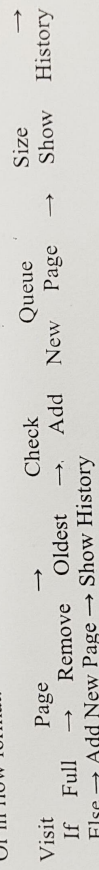
• 4. Algorithm (Pseudo Code)

1. Initialize a queue with maximum size N.
2. Start browsing process.
3. Input the visited webpage.
4. Check if the queue is full:
 - If yes, remove the oldest webpage (dequeue).
5. Add the new webpage to the queue (enqueue).
6. Display the current browser history.
7. Repeat steps 3–6 until browsing stops.
8. End the process.

5. System Architecture Flow

User Input → Queue Operation → Memory Update → Display Output

Or in flow format:



• 6. Data Flow Explanation

- Data enters the system as user input (visited URLs).
- The queue processes this data sequentially.
- Old data is removed when capacity is exceeded.
- Updated history is continuously reflected in the output.

- **7. Design Considerations**

- Fixed queue size for controlled memory usage
- Constant time operations for efficiency
- Sequential ordering to preserve navigation history
- Simplicity for easy implementation and understanding

Chapter 6

PRIOR ART AND RELATED WORK (LITERATURE SURVEY)

Browser history management has been widely studied in the domains of data structures and web systems. Early approaches primarily relied on simple linear data structures such as arrays and lists to store visited web pages. However, these methods often resulted in inefficiencies during operations like searching, updating, and deletion when the dataset became large.

Several research works have explored the use of fundamental data structures such as **stacks and queues** for managing ordered data. A stack follows the Last-In-First-Out (LIFO) principle and is commonly used in browser navigation for implementing backward and forward operations. Studies have shown that stack-based implementations allow efficient retrieval of recently visited pages and simplify navigation mechanisms.

For instance, a recent study implemented a browser history system using a stack within a web browser extension, demonstrating improved efficiency in storing, searching, and managing URLs compared to traditional list-based methods.

On the other hand, queues, which follow the First-In-First-Out (FIFO) principle, are widely used in applications requiring sequential processing and controlled memory usage. Research on data structures highlights that queues are effective for handling ordered data streams where older entries must be removed first, making them suitable for bounded history systems.

Additionally, studies on web navigation emphasize that managing browsing history efficiently is crucial for improving user experience, as users often need to revisit

previously accessed information. Various visualization and management techniques have been proposed to enhance navigation and usability.

Summary 1

While most existing systems and research focus on stack-based navigation or complex hybrid models, limited work emphasizes the use of a **pure queue-based approach** for browser history management. This project addresses that gap by implementing a simple, memory-efficient FIFO-based system, highlighting the practical applicability of queues in managing linear browsing history.

Chapter 7

APPLICATIONS AND DEPLOYMENT AREAS

1. Web Browser Systems

The proposed queue-based model can be used in simplified browser systems to manage recently visited pages efficiently. It ensures that only the most relevant and recent history is stored within a fixed memory limit.

2. Cache Management Systems

The FIFO mechanism is widely applicable in cache systems where older data must be replaced by newer data. This approach helps maintain updated and relevant information while controlling storage usage.

3. Embedded and Low-Resource Devices

In systems with limited memory, such as embedded devices or lightweight applications, the queue-based history model provides an efficient way to manage data without overloading system resources.

4. Mobile Applications

Mobile apps that track user activity or navigation history can use this approach to maintain recent records while automatically discarding outdated data to optimize performance.

5. Task Scheduling and Processing Systems

Queue-based logic is commonly used in scheduling tasks where operations must be processed in the order they arrive, making this model applicable beyond browser history management.

6. Educational Tools and Simulations

This project can be deployed as a teaching model to demonstrate the practical implementation of queue data structures in real-world scenarios for students and learners.

7. Logging and Monitoring Systems

Systems that maintain logs (such as system activity logs or event tracking) can use a fixed-size queue to store recent events while removing older entries automatically.

8. Prototype Systems for Browser Design

The model can serve as a foundational prototype for more advanced browser systems, where it can later be integrated with other data structures like stacks for full navigation functionality.

Chapter 8

CONCLUSION AND FUTURE SCOPE

Conclusion

This project successfully demonstrates the implementation of a browser history management system using the Queue data structure based on the FIFO (First-In-First-Out) principle. The system efficiently stores and manages visited web pages by maintaining a fixed-size history and automatically removing the oldest entries when the limit is exceeded. This ensures optimized memory usage and predictable system behavior.

The project highlights how fundamental data structures can be applied to solve practical problems in a simple and structured manner. Although real-world browser systems use more complex mechanisms, this implementation provides a clear understanding of sequential data handling and reinforces core concepts of queue operations such as enqueue and dequeue. Overall, the system achieves its objective of efficient and organized history management in a constrained environment.

Future Scope

- **Integration with Stack:**
Combine queue with stack to enable full browser navigation (backward and forward operations).
- **Graphical User Interface (GUI):**
Develop a user-friendly interface for better interaction and visualization of browser history.
- **Database Integration:**
Store history in databases for persistent storage and retrieval across sessions.
- **Dynamic Memory Management:**
Implement adaptive queue size based on system resources instead of fixed limits.

- **Search and Filtering Features:**
Add functionality to search and filter visited pages efficiently.
 - **Real Browser Simulation:**
Extend the system to simulate real-world browser behavior with tabs and sessions.
-

This gives your report a **strong ending with technical depth and future vision**, which evaluators look for.

References / Bibliography

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, "Introduction to Algorithms," 3rd ed., MIT Press, 2009.
- [2] S. Sahni, "Data Structures, Algorithms, and Applications in C++," 2nd ed., Universities Press, 2005.
- [3] GeeksforGeeks, "Queue Data Structure," 2024. [Online]. Available: <https://www.geeksforgeeks.org/queue-data-structure/>.
- [4] GeeksforGeeks, "Stack Data Structure," 2024. [Online]. Available: <https://www.geeksforgeeks.org/stack-data-structure/>.
- [5] MDN Web Docs, "Working with the History API," 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/History_API.
- [6] N. Wirth, "Algorithms + Data Structures = Programs," Prentice Hall, 1976.
- [7] A. S. Tanenbaum, "Modern Operating Systems," 3rd ed., Prentice Hall, 2008.

[8] ResearchGate, "A Comprehensive Analysis of Stack and Queue Data Structures," 2023. [Online].

Available: <https://www.researchgate.net/>

Appendices

Appendices

- **Appendix A: Source Code (Brief)**

```
from collections import deque
history = deque(maxlen=5)

history.append("google.com")
history.append("youtube.com")
history.append("chatgpt.com")
history.append("github.com")
history.append("stackoverflow.com")
history.append("wikipedia.org") # removes oldest

print(history)
```

- **Appendix B: Sample Input & Output**

Input: Google → YouTube → ChatGPT → GitHub → StackOverflow → Wikipedia

Output: youtube.com, chatgpt.com, github.com, stackoverflow.com, wikipedia.org

- **Appendix C: Test Case**

- Queue Limit = 5
- Input Pages = 6
- Result: Oldest page removed automatically

- **Appendix D: Requirements**

- **Hardware:** Basic computer
- **Software:** Python / C++ / Java